Práctica 6

Integración continua con Jenkins

Modalidad: instalación de Jenkins en Docker y pipeline guiado sobre un proyecto Maven (Parte A) + reto de diagnóstico con un pipeline y un proyecto con defectos sembrados (Parte B) + ejercicios propuestos (Parte C).

Duración estimada: 2 periodos de laboratorio.

Requisito: haber completado las Prácticas 1 a 5. Docker Desktop, Git y una cuenta de GitHub. Java 17 o superior y Maven 3.9 para probar en local.

Material: `practica6-proyectos.zip` con los proyectos `factorial-app` y `reto-ci`, y la carpeta `jenkins` con el `docker-compose.yml`.

Entregable: documento PDF con capturas, el historial de builds y la tabla de defectos corregidos + enlaces a los dos repositorios de GitHub, subidos a Classroom.

1. Objetivos

Al terminar la práctica el estudiante será capaz de:

- Explicar qué es la integración continua, qué problema resuelve y cómo orquesta las pruebas unitarias, el análisis estático, las pruebas E2E y las de rendimiento de las Prácticas 1 a 5.
- Levantar Jenkins en un contenedor Docker con un volumen persistente, desbloquearlo, instalar los plugins sugeridos y crear el usuario administrador.
- Configurar Maven como herramienta global de Jenkins y relacionar su nombre con el bloque `tools` del pipeline.
- Escribir un pipeline declarativo en un `Jenkinsfile` versionado junto al código, con etapas de compilación, pruebas y empaquetado.
- Crear un job de tipo Pipeline que lea el `Jenkinsfile` desde un repositorio de GitHub y se dispare solo ante cada commit.
- Publicar el informe de JUnit y archivar el artefacto `.jar`, y leer el resultado de un build: Stage View, Console Output, Test Result y Artifacts.
- Distinguir un build verde de un build sano: detectar un pipeline que no ejecuta pruebas, que ignora las que fallan o que no corre en el agente.
- Diagnosticar un build fallido localizando la capa responsable (configuración de Jenkins, `Jenkinsfile`, proyecto o código) y corregirla con un commit por defecto.

2. Requisitos previos

Herramienta	Versión	Cómo verificar / nota
Docker Desktop	4.x con WSL 2 (Windows)	`docker version` debe mostrar Client y Server. Si falla Server, Docker Desktop no está iniciado.

Herramienta	Versión	Cómo verificar / nota
Git	2.40 o superior	`git --version`. Configura `user.name` y `user.email` antes del primer commit.
Cuenta de GitHub	—	Los dos repositorios de la práctica serán públicos: así Jenkins los clona sin credenciales.
Java (JDK)	17 o superior	`java -version`. Solo para probar en local: dentro del contenedor Jenkins ya trae Java 17.
Maven	3.9.x	`mvn -v`. Solo en local; Jenkins descarga su propio Maven en el Paso 5.
Memoria RAM	4 GB libres	Jenkins usa entre 1 y 2 GB. Cierra la API y JMeter de la Práctica 5 mientras trabajas.
Espacio en disco	3 GB	La imagen pesa unos 500 MB; el volumen crece con los plugins, Maven y las dependencias.
Editor	VS Code o IntelliJ	Para editar el `Jenkinsfile`, el `pom.xml` y el código Java.

> **Sobre la descarga de la imagen:** la imagen `jenkins/jenkins:lts-jdk17` pesa unos 500 MB y los plugins sugeridos otros 150 MB. Si la conexión del laboratorio es lenta, el docente puede llevar la imagen en una memoria USB: `docker save jenkins/jenkins:lts-jdk17 -o jenkins.tar` en una máquina con conexión y `docker load -i jenkins.tar` en cada equipo. Los plugins se descargan igual durante el asistente, así que conviene hacer el Paso 4 al inicio de la clase.

3. Integración continua: qué es y dónde encaja

En las Prácticas 1 a 5 cada prueba se ejecutó cuando alguien se acordó de ejecutarla, en su propia máquina, con su propia versión de Java y sus propios archivos sin commitear. La integración continua cambia eso: **cada cambio que llega al repositorio se compila y se prueba automáticamente, en una máquina neutral, y el resultado es visible para todo el equipo en minutos**.

Jenkins no prueba nada por sí mismo. No sabe qué es un test ni qué es un percentil. Lo que hace es **ejecutar, en orden y sin que nadie lo pida, los comandos que ya conoces** (`mvn test`, `mvn sonar:sonar`, `npx cypress run`, `jmeter -n`) y decidir, a partir de lo que devuelven, si el cambio se acepta o se rechaza. Por eso un pipeline vale exactamente lo que valen las pruebas que ejecuta, y por eso un pipeline que no las ejecuta de verdad es peor que no tener ninguno: da una falsa seguridad.

Qué cambia con un servidor de CI

Pregunta	Sin integración continua	Con integración continua
¿Quién ejecuta las pruebas?	El desarrollador, si se acuerda.	Jenkins, en cada commit, siempre.
¿Dónde se ejecutan?	En la máquina de cada uno: “en mi máquina funciona”.	En un agente limpio, igual para todos.

Pregunta	Sin integración continua	Con integración continua
¿Cuándo se sabe que algo se rompió?	Días después, al integrar o en producción.	Minutos después del commit que lo rompió.
¿Quién lo rompió?	Hay que investigar entre muchos cambios.	El build rojo señala el commit exacto.
¿Cómo se construye el entregable?	A mano, cada vez un poco distinto.	Siempre igual, archivado y trazable al commit.

Cómo encaja con las prácticas anteriores

Práctica	Qué se automatizó	Comando	Etapa del pipeline
1 y 2 — JUnit y TDD	Pruebas unitarias	`mvn test`	Pruebas
3 — SonarQube	Análisis estático y Quality Gate	`mvn sonar:sonar`	Análisis (Ejercicio 5)
4 — Cypress	Pruebas E2E en navegador	`npx cypress run`	E2E
5 — JMeter	Pruebas de rendimiento contra un SLA	`jmeter -n -t plan.jmx`	Rendimiento (Ejercicio 5)
6 — Jenkins	Orquesta todo lo anterior en cada cambio	`Jenkinsfile`	El pipeline completo

Cada práctica anterior terminaba con un comando que "alguien" ejecutaba. Desde hoy ese alguien es Jenkins. En esta práctica el pipeline tiene tres etapas (compilar, probar, empaquetar); en los ejercicios de la Parte C le agregarás la cobertura, el análisis de SonarQube o la prueba de carga.

Cómo funciona Jenkins

Jenkins es un servidor web escrito en Java. Su trabajo es recibir un disparo (un commit, un clic, una hora del día), preparar un directorio de trabajo, ejecutar pasos y guardar lo que pasó. Cinco ideas bastan para entenderlo:

- **Controller y executors.** El controller es el Jenkins que ves en el navegador. Los executors son los "hilos" donde corren los builds. En esta práctica todo corre en el propio controller, dentro del contenedor Docker, que es un Linux: los comandos se ejecutan con `sh`, no con `bat`.
- **Job y build.** Un job es la definición de un trabajo (qué repositorio, qué pipeline). Un build es una ejecución concreta de ese job, numerada: #1, #2, #3. Cada build guarda su consola, sus resultados de pruebas y sus artefactos.
- **Pipeline como código.** El pipeline no se configura con clics: se escribe en un archivo `Jenkinsfile` que vive en la raíz del repositorio, junto al `pom.xml`. Así se versiona, se revisa y se revierte como cualquier otro código.
- **Workspace.** Es la carpeta donde Jenkins clona el repositorio para cada build. Todo lo que el pipeline genera (`target/`, informes) queda ahí.
- **Plugins.** Casi todo en Jenkins es un plugin: Git, Pipeline, JUnit, la vista de etapas. Los plugins sugeridos del asistente traen todo lo que usa esta práctica.

> **Pipeline como código:** la guía original de esta práctica pegaba el script del pipeline dentro de la configuración del job. Funciona, pero ese script no queda en el repositorio: nadie ve cuándo cambió ni quién lo cambió, y si se borra el job se pierde. En esta versión el pipeline se lee siempre desde el `Jenkinsfile` del repositorio (**Pipeline script from SCM**). Es lo que se hace en cualquier equipo real.

Anatomía de un Jenkinsfile declarativo

Bloque	Para qué sirve	En factorial-app
`pipeline { }`	Envuelve todo. Indica que es sintaxis declarativa.	Obligatorio.
`agent`	Dónde se ejecuta el pipeline.	`agent any`: en cualquier executor libre.
`tools`	Herramientas que Jenkins debe poner en el PATH. El nombre debe existir en Manage Jenkins → Tools.	`maven 'Maven-3.9'`
`options`	Comportamiento general del build.	Marcas de tiempo, conservar 10 builds, timeout de 15 min.
`triggers`	Qué dispara el build sin intervención humana.	`pollSCM('H/2 * * * *')`: revisa el repositorio cada 2 min.
`stages` / `stage`	Las etapas, en orden. Si una falla, las siguientes no se ejecutan.	Compilar, Pruebas, Empaquetar.
`steps`	Los comandos de cada etapa.	`sh 'mvn -B test'`
`post`	Lo que se ejecuta al terminar una etapa o el pipeline, según el resultado.	`junit` siempre; mensajes en `success`, `unstable` y `failure`.

Los resultados de un build

Resultado	Color	Qué significa	Causa típica
SUCCESS	Verde	Todas las etapas terminaron bien.	Todo correcto… si las pruebas se ejecutaron de verdad.
UNSTABLE	Amarillo	El pipeline terminó, pero hay pruebas que fallan.	El paso `junit` encontró fallos en los informes XML.
FAILURE	Rojo	Un paso devolvió un código de error y el pipeline se detuvo.	Error de compilación, `mvn test` con pruebas fallidas, herramienta inexistente.
ABORTED	Gris	Alguien lo detuvo o venció el timeout.	Un proceso colgado; el `timeout` de `options`.

La regla de la práctica: qué es un build sano

Así como en la Práctica 5 el SLA definía de antemano qué era "aceptable", aquí hace falta acordar qué significa que un build esté bien. **Verde no alcanza.** Un build sano cumple las siete condiciones siguientes, y contra ellas se evalúan los dos pipelines de la práctica:

Indicador	Condición	Dónde se lee
Resultado	SUCCESS	Página del build, icono y Stage View.
Pruebas ejecutadas	Las mismas que `mvn test` ejecuta en local, y más de cero	Enlace **Test Result** del build; línea `Tests run:` en la consola.
Pruebas fallidas	0 fallos y 0 errores	Test Result; `Failures: 0, Errors: 0`.
Informe publicado	Existe el enlace Test Result y el gráfico Test Result Trend del job	Página del build y del job.
Artefacto	El `.jar` está archivado y asociado al build	Sección **Build Artifacts** del build.
Disparo automático	Un commit en `main` inicia un build sin clic, en 3 minutos o menos	Causa del build: **Started by an SCM change**.
Reproducibilidad	Todo el pipeline está en el `Jenkinsfile`; el job solo tiene la URL del repositorio	Configuración del job: Pipeline script from SCM.

PARTE A — Práctica guiada: Jenkins en Docker y el pipeline de factorial-app

Paso 1. El material

Descomprime `practica6-proyectos.zip`. Contiene tres carpetas: `factorial-app` (el proyecto correcto con su `Jenkinsfile`, que usarás en toda la Parte A), `reto-ci` (otro proyecto con su pipeline, con defectos sembrados, para la Parte B) y `jenkins` (el `docker-compose.yml` para levantar Jenkins). No abras todavía la segunda.

`factorial-app` es el mismo ejemplo de la guía original de esta práctica, ordenado como en la Práctica 1: un proyecto Maven con una clase `Factorial` y sus pruebas JUnit 5.

Archivo	Qué contiene
`pom.xml`	Java 17, JUnit 5.11 y Surefire 3.5. Igual que en la Práctica 1.
`src/main/java/bo/edu/usfx/factorial/Factorial.java`	`calcular(int)`: lanza `IllegalArgumentException` si el número es negativo o mayor que 20 (21! no cabe en un `long`).
`src/test/java/bo/edu/usfx/factorial/FactorialTest.java`	9 pruebas: 0!, 5!, cinco factoriales conocidos parametrizados, negativo y desborde.
`Jenkinsfile`	El pipeline declarativo: Compilar, Pruebas (con informe JUnit) y Empaquetar (con el `.jar` archivado).
`.gitignore`	Excluye `target/` y archivos del IDE.

Paso 2. Probar en local antes de automatizar

Jenkins va a ejecutar exactamente los mismos comandos que tú. Si no funcionan en tu máquina, no van a funcionar en Jenkins. Abre una terminal en `factorial-app`:

```
cd factorial-app
mvn -B clean test
```

Al final debe aparecer `Tests run: 9, Failures: 0, Errors: 0, Skipped: 0` y `BUILD SUCCESS`. **Anota el número 9**: es el número de pruebas que el build de Jenkins tiene que mostrar. Captura la salida: es la **Captura 1** del entregable.

Paso 3. Levantar Jenkins en Docker

Con Docker Desktop iniciado, abre una terminal en la carpeta `jenkins` del material y levanta el contenedor:

```
cd jenkins
docker compose up -d
docker ps
```

`docker ps` debe mostrar el contenedor `jenkins-com450` con estado `Up` y el puerto `0.0.0.0:8090->8080/tcp`. Si prefieres no usar compose, el comando equivalente es:

```
docker pull jenkins/jenkins:lts-jdk17
docker run -d --name jenkins-com450 -p 8090:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home --restart unless-stopped jenkins/jenkins:lts-jdk17
```

Opción	Qué hace
`-d`	Ejecuta el contenedor en segundo plano.
`--name jenkins-com450`	Nombre fijo para referirse al contenedor en los demás comandos.
`-p 8090:8080`	El puerto 8080 del contenedor se publica como 8090 en tu máquina. Usamos 8090 porque la API de la Práctica 5 ocupa el 8080.
`-p 50000:50000`	Puerto para agentes remotos. No se usa hoy, pero es el estándar.
`-v jenkins_home:/var/jenkins_home`	Volumen con toda la configuración, los jobs y los builds.
`--restart unless-stopped`	Jenkins vuelve a arrancar solo cuando reinicias Docker Desktop.

> **Por qué un volumen:** un contenedor es desechable: si lo borras, se pierde todo lo que tenía dentro. Con el volumen `jenkins_home`, puedes hacer `docker rm -f jenkins-com450` y volver a crearlo con el mismo comando: los plugins, el usuario, los jobs y el historial de builds siguen ahí. Sin volumen, cada vez que el contenedor se recrea hay que repetir los Pasos 4 y 5.

Paso 4. Desbloquear Jenkins e instalar los plugins

Abre `http://localhost:8090`. La primera vez Jenkins tarda uno o dos minutos en arrancar y luego pide la contraseña inicial de administrador. Obtenla del contenedor:

```
docker exec jenkins-com450 cat /var/jenkins_home/secrets/initialAdminPassword
```

1. Pega la contraseña en **Unlock Jenkins** y pulsa Continue. La contraseña es distinta en cada instalación: no sirve la de otro compañero ni la de una guía.
2. En **Customize Jenkins** elige **Install suggested plugins**. Tarda entre 3 y 10 minutos según la conexión. Trae, entre otros, Git, Pipeline, Pipeline: Stage View, JUnit y Timestamper.
3. En **Create First Admin User** crea tu usuario (no sigas como admin con la contraseña inicial). Anota el usuario y la clave.
4. En **Instance Configuration** deja `http://localhost:8090/` y pulsa Save and Finish, luego Start using Jenkins.

Debes ver el panel principal con el mensaje Welcome to Jenkins!. Captúralo con tu nombre de usuario visible arriba a la derecha: es la **Captura 2**.

> **Si un plugin queda con una cruz roja:** casi siempre es un corte de red durante la descarga. Pulsa Retry. Si persiste, continúa, entra a Manage Jenkins → Plugins → Available plugins e instala el que falte (por nombre) y reinicia con `docker restart jenkins-com450`.

Paso 5. Configurar Maven como herramienta

El contenedor trae Java 17 pero no Maven. En lugar de instalarlo a mano dentro del contenedor, se declara como herramienta y Jenkins lo descarga la primera vez que un pipeline lo pide:

1. Manage Jenkins → **Tools**.
2. Baja hasta **Maven installations** y pulsa Add Maven.
3. Name: `Maven-3.9` (exactamente así: mayúscula, guion, sin espacios).
4. Deja marcado **Install automatically** y en Version elige la 3.9 más reciente (3.9.11 al momento de escribir esta guía).
5. Pulsa **Save** al final de la página.

No hace falta configurar un JDK: si el pipeline no pide uno, Jenkins usa el Java 17 del contenedor, que es la versión para la que compila el `pom.xml`.

> **El nombre es un contrato:** el `Jenkinsfile` dice `maven 'Maven-3.9'`. Jenkins busca una instalación con ese nombre exacto; si no la encuentra, el build falla antes de ejecutar la primera etapa. El nombre no es la versión: podrías llamarla `mvn-del-labo` siempre que el pipeline use el mismo texto. Recuerda esto en la Parte B.

Paso 6. Subir factorial-app a GitHub

En GitHub crea un repositorio **público** y vacío llamado `factorial-app` (sin README, sin .gitignore, sin licencia: ya vienen en el material). Luego, desde la carpeta del proyecto:

```
cd factorial-app
git init -b main
git add .
git commit -m "Proyecto factorial con Jenkinsfile"
git remote add origin https://github.com/TU-USUARIO/factorial-app.git
git push -u origin main
```

Comprueba en GitHub que el `Jenkinsfile` está en la raíz del repositorio, al mismo nivel que el `pom.xml`, y que la carpeta `target/` no se subió.

> **Repositorio privado:** si tu repositorio es privado, Jenkins necesita credenciales. Crea en GitHub un Personal Access Token (Settings → Developer settings → Fine-grained tokens) con permiso de solo lectura sobre el repositorio, y regístralo en Manage Jenkins → Credentials como Username with password (usuario de GitHub + token). Nunca uses tu contraseña de GitHub ni pegues el token en el `Jenkinsfile`.

Paso 7. Crear el job de tipo Pipeline

1. En el panel principal: **New Item**. Nombre: `factorial-app`. Tipo: **Pipeline**. OK.
2. En la sección **Pipeline**, Definition: **Pipeline script from SCM**.
3. SCM: **Git**. Repository URL: `https://github.com/TU-USUARIO/factorial-app.git`. Credentials: ninguna (repositorio público).
4. Branch Specifier: `*/main` (el valor por defecto es `*/master`: cámbialo, o Jenkins no encontrará la rama).
5. Script Path: `Jenkinsfile`. Pulsa **Save**.
6. Pulsa **Build Now** en el menú de la izquierda.

Fíjate en lo que **no** configuraste: ni las etapas, ni los comandos, ni el disparo automático, ni el informe de pruebas. Todo eso está en el `Jenkinsfile`. El job solo sabe dónde está el repositorio.

Paso 8. Leer el build

El primer build tarda entre 2 y 5 minutos: Jenkins descarga Maven y después Maven descarga las dependencias al repositorio local del contenedor. Los siguientes tardan menos de un minuto. Cuando termine, recorre estas pantallas:

Pantalla	Dónde	Qué debes ver en factorial-app
Stage View	Página del job	Tres columnas verdes: Compilar, Pruebas, Empaquetar, con su duración.
Console Output	Build #1 → Console Output	`Unpacking … apache-maven-3.9.11-bin.zip` la primera vez, `Tests run: 9, Failures: 0`, `Recording test results`, `Archiving artifacts` y `Finished: SUCCESS`.
Test Result	Build #1 → Test Result	9 pruebas, 0 fallos. Entra hasta `FactorialTest` y mira cada caso, incluidos los parametrizados.
Build Artifacts	Página del build	`factorial-app-1.0-SNAPSHOT.jar`, descargable.
Changes	Build → Changes	El commit que se construyó, con autor y mensaje.
Test Result Trend	Página del job	Aparece a partir del segundo build con informe.

Captura la página del job con la Stage View en verde: **Captura 3**. Captura el Test Result con las 9 pruebas: **Captura 4**.

Compara el número de pruebas con el que anotaste en el Paso 2. Tienen que coincidir. Esta comprobación, que parece trivial, es la que en la Parte B separa un build sano de uno que miente.

Paso 9. Romper el código y ver el build en rojo

Un pipeline que nunca se pone rojo no demuestra nada. Introduce un defecto a propósito en `Factorial.java`: cambia la multiplicación por una suma.

```
// ANTES
resultado *= i;
// DESPUES (defecto introducido a proposito)
resultado += i;

git commit -am "Rompe el factorial a proposito"
git push
```

No pulses Build Now. Espera: en menos de dos minutos Jenkins detecta el commit y lanza el build solo (Paso 10). El build termina en **FAILURE** (rojo): la etapa Pruebas falla porque `mvn test` devuelve error, y la etapa Empaquetar aparece como omitida en la Stage View. Aun así el informe se publica, porque el paso `junit` está en un bloque `post { always { } }`. Abre Test Result: 4 de las 9 pruebas fallan (5!, 2!, 10! y 20!; 0!, 1! y 3! siguen pasando por casualidad, porque 1 + 2 + 3 = 3!), y cada una muestra el valor esperado y el obtenido, por ejemplo `expected: <120> but was: <15>`. Vale la pena notarlo: con una sola prueba de 3! este defecto habría pasado.

Revierte el cambio, haz commit y push, y espera el build verde siguiente. Captura la Stage View con el historial rojo → verde: **Captura 5**.

> **Rojo o amarillo:** aquí el build queda en rojo porque `mvn test` termina con error y eso detiene el pipeline. Si `mvn` no fallara (por ejemplo, con `-Dmaven.test.failure.ignore=true`), el paso `junit` marcaría el build en amarillo (UNSTABLE) al leer los fallos. Los dos son aceptables como señal; lo que nunca es aceptable es que el build quede verde con pruebas fallando. Vas a ver ese caso en la Parte B.

Paso 10. El disparo automático

El build del Paso 9 se lanzó sin que nadie hiciera clic. Lo hizo el bloque `triggers` del `Jenkinsfile`:

```
triggers {
    pollSCM('H/2 * * * *')
}
```

- La expresión tiene la sintaxis de cron (minuto, hora, día del mes, mes, día de la semana). `H/2 * * * *` significa “cada 2 minutos”.
- La `H` (hash) reparte la carga: Jenkins elige un desfase fijo por job para que no todos consulten GitHub en el mismo segundo.
- Jenkins registra el trigger la primera vez que ejecuta el `Jenkinsfile`. Por eso el primer build siempre es manual (Build Now).
- En el job aparece la opción **Git Polling Log**: muestra cada consulta y si encontró cambios.

Abre el build del Paso 9: arriba debe decir **Started by an SCM change**. Captura esa página: **Captura 6**.

> **¿Por qué polling y no webhook?** lo ideal es que GitHub avise a Jenkins en el instante del push (webhook). Pero GitHub está en Internet y tu Jenkins en `localhost`: GitHub no puede alcanzarlo. El polling invierte la dirección: es Jenkins el que pregunta. En el Ejercicio 4 de la Parte C harás funcionar el webhook con un túnel.

Paso 11. Cómo se interpreta un build fallido

Un build rojo no se arregla mirando el icono. Se lee la consola de arriba hacia abajo buscando la **primera** línea de error, porque todas las siguientes suelen ser consecuencia de esa. Luego se ubica el problema en una de cuatro capas, de afuera hacia adentro:

1. **Configuración de Jenkins.** Una herramienta, una credencial o un plugin que no existe. El build falla en segundos, a veces antes de la primera etapa. Se corrige en Manage Jenkins o en el nombre que usa el pipeline.
2. **El Jenkinsfile.** Un paso que no existe en ese agente, una etapa mal ordenada, una opción que oculta errores. Se corrige con un commit al `Jenkinsfile`.
3. **El proyecto.** El `pom.xml`, la estructura de carpetas, los nombres de las clases de prueba. El mismo problema aparece en local con `mvn test`.
4. **El código.** Una prueba que falla porque el código está mal. El informe de JUnit dice cuál y por qué.

La pregunta que decide entre las dos primeras capas y las dos últimas es siempre la misma: **¿se reproduce en local con el mismo comando?** Si `mvn -B test` falla igual en tu máquina, el problema es del proyecto o del código. Si en tu máquina funciona y en Jenkins no, el problema es del entorno o del pipeline.

PARTE B — Reto guiado: el pipeline que miente

Hasta aquí construiste un pipeline que funciona. En esta parte recibes uno que alguien más escribió: el proyecto `calculadora-descuentos`, que implementa el ejercicio de la guía original de esta práctica, con su `Jenkinsfile`. El equipo anterior asegura que "el pipeline está en verde". **La carpeta `reto-ci` tiene seis defectos sembrados**: dos en la forma en que el pipeline se relaciona con Jenkins, dos que hacen que el build diga verde sin que las pruebas protejan nada, y dos en el código que esos dos últimos escondían.

Las reglas de negocio de `CalculadoraDescuentos.calcularPrecioFinal(precioOriginal, porcentajeDescuento)` son las mismas de la guía original:

Regla	Detalle
Porcentaje inválido	Menor que 0 o mayor que 100 → `IllegalArgumentException`.
Precio inválido	Menor o igual que 0 → `IllegalArgumentException` con el mensaje "El precio original debe ser mayor que cero".
Redondeo	El precio final se redondea a dos decimales, la mitad hacia arriba: 29,997 → 30,00.
Descuento 0	El precio final es igual al original.
Descuento 100	El precio final es 0.

Paso 1. Subir el reto y crear su job

Crea en GitHub un repositorio público `calculadora-descuentos` y sube el contenido de `reto-ci` igual que en el Paso 6 de la Parte A. Crea un job Pipeline llamado `reto-ci` igual que en el Paso 7, apuntando a ese repositorio. Pulsa Build Now.

> **No toques la configuración de Jenkins para que el reto pase:** no crees una herramienta nueva con otro nombre, no cambies el job a un script pegado, no instales plugins. Todos los defectos se corrigen con commits al repositorio. Cada corrección es un commit separado con un mensaje que diga qué defecto corrige: el historial de builds del job es la evidencia de tu diagnóstico.

Paso 2. Los síntomas

Los defectos no aparecen todos a la vez: cada uno esconde al siguiente. Esta es la secuencia que vas a observar si corriges uno por build (valores del entorno del docente):

Build	Síntoma	Dónde se ve
1	FAILURE en pocos segundos, sin que se ejecute ninguna etapa.	Console Output: `WorkflowScript: 6: Tool type "maven" does not have an install of "Maven 3.8.5" configured.`
2	FAILURE en la etapa Compilar. Hay que lanzarlo con Build Now (ver Paso 4).	Console Output: `java.io.IOException: Batch scripts can only be run on Windows nodes.`
3	**SUCCESS**. Tres etapas verdes, `.jar` archivado. Pero la página del build no tiene enlace Test Result.	Consola de la etapa Pruebas: no aparece ninguna línea `Tests run:`.
4	**SUCCESS** otra vez. Ahora la consola dice `Tests run: 13, Failures: 1, Errors: 1` y aun así `BUILD SUCCESS`.	Console Output, etapa Pruebas.
5	FAILURE con Test Result: 2 de 13 pruebas en rojo. La etapa Empaquetar ya no se ejecuta.	Test Result: `descuentoTotalDejaElPrecioEnCero` y `aplicaElDescuentoYRedondeaADosDecimales[4]`.
6	FAILURE con 1 prueba en rojo.	Test Result: queda solo una de las dos anteriores.
7	SUCCESS con 13 pruebas, 0 fallos, `.jar` archivado: build sano.	Test Result, Stage View, Artifacts.

Fíjate en los builds 3 y 4: **son verdes**. Si el equipo anterior miraba solo el color, tenía razón: “el pipeline está en verde”. Contra la regla del build sano de la Sección 3 incumplen dos indicadores cada uno. Antes de tocar nada, completa esta tabla en tu informe, una fila por defecto:

Columna	Qué escribir
Síntoma observado	Qué build, qué etapa y qué línea de la consola, tal como aparece.
¿Se reproduce en local?	Sí o no, con el comando que usaste. Decide la capa.
Capa	Configuración de Jenkins, `Jenkinsfile`, proyecto o código (Parte A, Paso 11).

Columna	Qué escribir
Indicador del build sano o regla violada	Cuál de las siete filas de la Sección 3 o de las cinco reglas de negocio se incumple.
Archivo y línea	Dónde está la causa.
Corrección aplicada	La línea antes y después, el hash del commit y el número del build que lo confirma.

Paso 3. Método de trabajo: cuatro preguntas

Para cada síntoma, responde en este orden y no saltes ninguna:

1. **¿Qué etapa falla y cuál es la primera línea de error?** No digas "el build falla": di "el build #2 falla en la etapa Compilar con `Batch scripts can only be run on Windows nodes`".
2. **¿Se reproduce en local con el mismo comando?** Ejecuta en tu máquina lo que ejecuta la etapa. Si en local funciona, el problema está en el entorno o en el pipeline; si falla igual, en el proyecto o en el código.
3. **¿En qué capa está?** Configuración de Jenkins, `Jenkinsfile`, proyecto o código. La respuesta dice qué archivo abrir.
4. **¿Cuál es la corrección mínima?** Cambia lo justo, haz un commit, deja que el polling lance el build y confirma que el síntoma desapareció y que no apareció otro que no esperabas.

> **Un defecto, un commit, un build:** los seis defectos se encadenan: mientras el pipeline no llegue a la etapa Pruebas, no sabrás si las pruebas corren; mientras las pruebas no corran, no sabrás si fallan; mientras el build ignore los fallos, no sabrás cuáles son. Si corriges tres cosas en un commit y el build sale verde, no puedes demostrar cuál de las tres era necesaria.

Paso 4. Defecto 1 resuelto — la herramienta que no existe

Este va resuelto como ejemplo del método. El build #1 termina en unos segundos. La Stage View no muestra ninguna etapa y la consola termina así:

```
Checking out Revision 4f4f281… (refs/remotes/origin/main)
Commit message: "inicial"
First time build. Skipping changelog.
org.codehaus.groovy.control.MultipleCompilationErrorsException: startup failed:
WorkflowScript: 6: Tool type "maven" does not have an install of "Maven 3.8.5"
configured - did you mean "Maven-3.9"? @ line 6, column 15.
           maven 'Maven 3.8.5'
                 ^

1 error

Finished: FAILURE
```

Pregunta 1 — etapa y primera línea. No llegó a ninguna etapa. Jenkins clonó el repositorio, leyó el `Jenkinsfile` y falló al **compilarlo**: `WorkflowScript: 6` es la línea 6 del `Jenkinsfile`, y `startup failed` significa que el

pipeline ni siquiera arrancó. La línea que importa es `Tool type "maven" does not have an install of "Maven 3.8.5" configured`.

Pregunta 2 — ¿se reproduce en local? No: en local `mvn -B clean compile` funciona. En local no hay bloque `tools`; Maven está en el PATH. El problema es del entorno o del pipeline.

Pregunta 3 — ¿qué capa? El `Jenkinsfile` pide una instalación llamada `Maven 3.8.5`, que es el nombre que usaba la guía original. En este Jenkins la instalación se llama `Maven-3.9` (Parte A, Paso 5). El mismo Jenkins lo sugiere: did you mean "Maven-3.9"?. Hay dos correcciones posibles: crear en Jenkins una herramienta con ese nombre, o cambiar el nombre en el `Jenkinsfile`. La regla del reto descarta la primera, y con razón: si cada proyecto pide su propio nombre, el servidor termina con diez instalaciones de Maven idénticas.

```
// ANTES (defectuoso)
tools {
    maven 'Maven 3.8.5'
}
// DESPUES (corregido)
tools {
    maven 'Maven-3.9'
}
```

Pregunta 4 — corrección mínima. Una línea del `Jenkinsfile`. Commit con el mensaje `Defecto 1: nombre de la herramienta Maven` y push. Esta vez **el build no arranca solo**: como el `Jenkinsfile` nunca llegó a ejecutarse, Jenkins todavía no conoce su bloque `triggers`. Lanza el build #2 con Build Now; desde ahí el polling queda registrado y los siguientes arrancan con cada push. El build #2 falla en otro lugar, y eso es lo esperado: el defecto 1 escondía al 2.

> **Lo que acaba de pasar:** este defecto no lo detecta ninguna prueba de las Prácticas 1 a 5, porque no está en el código: está en el contrato entre el pipeline y el servidor. Por eso se lee la consola y no solo el icono, y por eso la primera pregunta siempre es “¿en local funciona?”.

Paso 5. Los cinco defectos restantes

Cada síntoma de la tabla del Paso 2 corresponde a un defecto distinto. Estas pistas te dicen dónde mirar, no qué escribir.

Síntoma	Pista
Build #2: `Batch scripts can only be run on Windows nodes` en la etapa Compilar.	¿En qué sistema operativo corre el agente? El contenedor de Jenkins es Linux. Mira qué paso usa cada etapa para ejecutar Maven y compáralo con el `Jenkinsfile` de `factorial-app`. Son las tres etapas, no solo la primera.
Build #3 verde sin Test Result y sin ninguna línea `Tests run:`.	Ejecuta `mvn -B test` en local: tampoco corre ninguna prueba, y Maven no se queja. Surefire solo ejecuta las clases cuyo nombre sigue ciertos patrones (Anexo C). Mira el nombre de la clase de prueba. Hay dos correcciones posibles; elige la que no obligue a configurar nada.

Síntoma	Pista
Build #4 verde con `Failures: 1, Errors: 1` en la consola.	Lee el comando de la etapa Pruebas completo, hasta el final. Hay una opción que le dice a Maven que no falle aunque fallen las pruebas. Y falta algo que el pipeline de `factorial-app` sí tiene en esa etapa: sin eso, Jenkins nunca se entera de que hubo fallos. Son dos cambios en la misma etapa.
`descuentoTotalDejaElPrecioEnCero` lanza `IllegalArgumentException`.	Regla "Descuento 100". Mira la validación del porcentaje en `calcularPrecioFinal`: ¿qué valores rechaza exactamente? Compara con "menor que 0 o mayor que 100".
`33.33` con `10 %` da `29.99` y se esperaba `30.00`.	Regla "Redondeo". Mira el método `redondear`: ¿redondea o trunca? Un cast a `int` corta los decimales. `Math.round` o `BigDecimal` con `RoundingMode.HALF_UP` hacen lo que pide la regla.

Verificación de la Parte B: cuando el job `reto-ci` cumpla las siete filas del build sano (SUCCESS, 13 pruebas ejecutadas, 0 fallos, informe publicado, `.jar` archivado, build iniciado por un cambio en el repositorio, todo en el `Jenkinsfile`), captura la página del job con la Stage View y el Test Result Trend: es la **Captura 7**. Y anota el dato importante: **no cambiaste ni una prueba**. El reto se resuelve corrigiendo el pipeline y el código, no ajustando las pruebas para que pasen.

PARTE C — Ejercicios propuestos

Ejercicio 1 — Terminar el reto

Corrige los cinco defectos restantes de `reto-ci` hasta que su build cumpla las siete condiciones del build sano. Documenta cada uno en la tabla de seis columnas del Paso 2 de la Parte B, con el fragmento antes y después, el hash del commit y el número del build que confirma la corrección.

Ejercicio 2 — La cobertura como puerta de calidad

Un build puede tener 13 pruebas en verde y aun así dejar sin probar la mitad del código. Agrega al `pom.xml` de `calculadora-descuentos` el plugin `jacoco-maven-plugin` (versión 0.8.12 o superior) con tres ejecuciones: `prepare-agent`, `report` en la fase `verify` y `check` en la fase `verify` con una regla de cobertura de líneas mínima de 0,80 sobre el `BUNDLE`.

Agrega una etapa `Cobertura` después de Pruebas que ejecute `mvn -B verify` y archive la carpeta `target/site/jacoco/` con `archiveArtifacts`. Demuestra que la puerta funciona: agrega a `CalculadoraDescuentos` un método público sin pruebas (por ejemplo, un descuento por cantidad) hasta que la cobertura baje de 80 %, y captura el build rojo con el mensaje `Coverage checks have not been met`. Después escribe las pruebas que faltan y captura el build verde.

Ejercicio 3 — Ramas y Multibranch Pipeline

Crea un job de tipo **Multibranch Pipeline** llamado `descuentos-multibranch` apuntando al repositorio `calculadora-descuentos` (Branch Sources → Git; Scan Multibranch Pipeline Triggers → Periodically if not otherwise run, 1 minuto). Jenkins crea un job por cada rama que tenga `Jenkinsfile`.

Crea la rama `feature/descuento-por-cantidad`, escribe primero una prueba que falle (TDD, como en la Práctica 2), súbela y comprueba que la rama queda en rojo mientras `main` sigue en verde. Termina la funcionalidad, une la rama con `main` y entrega capturas de las dos ramas en cada estado. Responde en un párrafo: ¿qué protege este esquema que un único job sobre `main` no protege?

Ejercicio 4 — Webhook en lugar de polling

Mide cuánto tarda el polling: haz cinco commits separados y anota para cada uno la hora del push y la hora de inicio del build. Luego expón tu Jenkins con un túnel (`ngrok http 8090` o `smee.io` con su cliente), configura en GitHub un webhook (Settings → Webhooks, Payload URL `https://…/github-webhook/`, content type `application/json`, evento push) y cambia el trigger del `Jenkinsfile` por `githubPush()`. Repite los cinco commits.

Entrega las dos tablas de tiempos y responde: ¿qué ventaja tiene cada mecanismo?, ¿qué riesgo de seguridad introduce el túnel y cómo lo mitigarías? Apaga el túnel al terminar.

> **Cuidado con el túnel:** mientras esté activo, cualquiera con la URL puede ver la pantalla de login de tu Jenkins. Asegúrate de haber creado un usuario con contraseña robusta en el Paso 4 y no dejes el túnel abierto fuera de la clase.

Ejercicio 5 — Integrar una práctica anterior

Agrega al pipeline de `calculadora-descuentos` una etapa que automatice una de las prácticas anteriores. Elige una:

- **Análisis estático (Práctica 3).** Con SonarQube corriendo en tu máquina en el puerto 9000, agrega una etapa `Análisis` que ejecute `mvn -B sonar:sonar -Dsonar.host.url=http://host.docker.internal:9000` con el token guardado como credencial de tipo Secret text y leído con `withCredentials`. Nunca pegues el token en el `Jenkinsfile`. Desde dentro del contenedor, `localhost` es el propio contenedor: por eso se usa `host.docker.internal`.
- **Rendimiento (Práctica 5).** Con la API `api-usfx` corriendo en tu máquina en el puerto 8080, agrega una etapa `Rendimiento` que descargue JMeter 5.6.3 al workspace (solo si no existe), ejecute `01-smoke-login.jmx` en modo no-GUI contra `host.docker.internal` y falle el build si la línea `summary =` reporta errores. Archiva el `.jtl` y el dashboard HTML.

Entrega el `Jenkinsfile` final, una captura de la Stage View con la nueva etapa en verde y otra con la etapa en rojo provocada a propósito (un code smell grave para SonarQube, o la API detenida para JMeter).

Ejercicio 6 — Análisis escrito

Media página por pregunta, con ejemplos concretos de tu propio trabajo en esta práctica:

1. Los builds #3 y #4 del reto fueron verdes. ¿Qué habría pasado en un equipo que solo mira el color del build? ¿Qué indicador del build sano los habría detectado primero?
2. De los seis defectos del reto, ¿cuáles habría detectado un desarrollador ejecutando `mvn test` en su máquina antes del push, y cuáles solo aparecen en el servidor de integración? ¿Por qué?
3. El defecto 3 (pruebas que no se ejecutan) es invisible en local y en Jenkins. ¿Qué práctica de equipo o qué control automático propondrías para que no vuelva a pasar? Relaciónalo con el TDD de la Práctica 2.

4. ¿Qué política propondrías sobre cuándo se permite ignorar un fallo en el pipeline (`failure.ignore`, `allowEmptyResults`, subir un umbral)? Relaciónalo con lo discutido en la Práctica 3 sobre apagar reglas, en la Práctica 4 sobre modificar specs y en la Práctica 5 sobre subir umbrales de rendimiento.

4. Entregable

Sube a Classroom, dentro de la carpeta `practica6` de tu repositorio:

- PDF con las Capturas 1 a 7 (`mvn test` en local, panel de Jenkins con tu usuario, Stage View de `factorial-app`, Test Result con 9 pruebas, historial rojo → verde, build iniciado por un cambio en el repositorio, `reto-ci` sano).
- En el mismo PDF: la tabla de seis columnas con los seis defectos del reto y su corrección, con código antes y después, hash del commit y número de build.
- En el mismo PDF: las capturas y respuestas de los Ejercicios 2 a 5 y el análisis escrito del Ejercicio 6.
- Los enlaces a tus dos repositorios públicos: `factorial-app` y `calculadora-descuentos`. El historial de commits de este último debe mostrar un commit por defecto.

> **Verificación final:** antes de entregar, haz un último commit trivial en cada repositorio (por ejemplo, una línea en el README) y comprueba que los dos jobs arrancan solos y terminan en SUCCESS con el número de pruebas correcto. Si para que el reto pasara modificaste o borraste pruebas, creaste herramientas con otro nombre en Jenkins o pegaste el pipeline en el job, el reto no cuenta.

5. Rúbrica de evaluación

Criterio	Puntos
Jenkins en Docker con volumen, desbloqueado, plugins instalados y usuario propio (Capturas 1 y 2)	10
Maven configurado como herramienta; job `factorial-app` desde SCM en verde con 9 pruebas y `.jar` archivado (Capturas 3 y 4)	15
Ciclo rojo → verde provocado con commits y disparo automático por polling (Capturas 5 y 6)	10
Calidad del pipeline: todo en el `Jenkinsfile`, etapas con nombre, `junit` en `post always`, commits con mensajes claros	5
Los 6 defectos de `reto-ci` corregidos: build sano sin modificar pruebas ni la configuración de Jenkins (Captura 7)	20
Tabla de diagnóstico de los 6 defectos, completa, con capa, commit y número de build	10
Ejercicio 2: puerta de cobertura con JaCoCo, build rojo y verde	8
Ejercicio 3: Multibranch con una rama en rojo y `main` en verde	7
Ejercicio 4: tablas de tiempos polling vs. webhook y análisis de seguridad	5
Ejercicio 5: etapa de análisis estático o de rendimiento integrada	5
Ejercicio 6: análisis escrito y presentación del informe	5

Criterio	Puntos
TOTAL	**100**

> **Cómo se pierden puntos rápido:** entregar un pipeline verde que no ejecuta pruebas, modificar o borrar pruebas del reto, crear en Jenkins una herramienta llamada `Maven 3.8.5`, corregir los seis defectos en un solo commit, pegar tokens o contraseñas en el `Jenkinsfile`, o subir la carpeta `target/` al repositorio.

6. Solución de problemas frecuentes

Síntoma	Causa probable y solución
`docker: error during connect` o `Cannot connect to the Docker daemon`	Docker Desktop no está iniciado. Ábrelo y espera a que diga Engine running.
`Bind for 0.0.0.0:8090 failed: port is already allocated`	Otro programa usa el 8090. Cambia `8090:8080` por `8091:8080` y usa ese puerto en el navegador.
`Conflict. The container name "/jenkins-com450" is already in use`	Ya lo creaste. Usa `docker start jenkins-com450` en lugar de `docker run`.
El navegador muestra Jenkins is getting ready to work… varios minutos	Normal la primera vez. Si pasa de 5 minutos, mira `docker logs jenkins-com450`.
No encuentro la contraseña inicial	`docker exec jenkins-com450 cat /var/jenkins_home/secrets/initialAdminPassword`. También aparece en `docker logs`.
`Couldn't find any revision to build` al clonar	El Branch Specifier dice `*/master` y tu rama es `main`. Cámbialo en la configuración del job.
`Authentication failed` o `repository not found` al clonar	El repositorio es privado o la URL tiene un error. Hazlo público o registra un token (Parte A, Paso 6).
`Tool type "maven" does not have an install of …`	El nombre del bloque `tools` no coincide con Manage Jenkins → Tools. Deben ser idénticos.
`Batch scripts can only be run on Windows nodes`	El `Jenkinsfile` usa `bat` y el agente es Linux. Usa `sh`.
`WorkflowScript: N: …` y ninguna etapa en la Stage View	El `Jenkinsfile` no compila: un error de sintaxis o una herramienta inexistente en la línea N. Como no llegó a ejecutarse, el trigger no se registra: después de corregirlo, lanza el build con Build Now.
`release version 21 not supported` o `invalid target release`	El `pom.xml` pide una versión de Java mayor que la del contenedor (17). Usa `maven.compiler.release` 17.
`No test report files were found. Configuration error?`	El paso `junit` no encontró XML: las pruebas no se ejecutaron o la ruta no es `target/surefire-reports/*.xml`.
El commit no dispara ningún build	El trigger se registra en el primer build: lanza uno con Build Now. Revisa Git Polling Log.

Síntoma	Causa probable y solución
El primer build tarda mucho en `Downloading from central`	Maven descarga las dependencias al volumen. Solo pasa la primera vez por proyecto.
El contenedor se reinició y se perdió todo	Se creó sin `-v jenkins_home:/var/jenkins_home`. Recréalo con el volumen y repite los Pasos 4 y 5.

Anexo A — Comandos usados en la práctica

Comando	Qué hace
`docker compose up -d`	Levanta Jenkins con la configuración de la carpeta `jenkins`.
`docker ps`	Contenedores en ejecución y sus puertos.
`docker exec jenkins-com450 cat /var/jenkins_home/secrets/initialAdminPassword`	Contraseña inicial de administrador.
`docker logs -f jenkins-com450`	Registro de Jenkins en vivo (Ctrl+C para salir).
`docker stop jenkins-com450` / `docker start jenkins-com450`	Detiene y vuelve a arrancar sin perder datos.
`docker restart jenkins-com450`	Reinicia Jenkins (tras instalar plugins).
`docker rm -f jenkins-com450 && docker volume rm jenkins_home`	Borra el contenedor y todos sus datos. Solo para empezar de cero.
`mvn -B clean test`	Compila y ejecuta las pruebas en local, en modo no interactivo (`-B`), igual que Jenkins.
`mvn -B -DskipTests package`	Genera el `.jar` sin volver a ejecutar las pruebas.
`git init -b main` / `git remote add origin URL` / `git push -u origin main`	Crea el repositorio local con rama `main` y lo sube a GitHub.
`git commit -am "mensaje"` y `git push`	Registra y sube un cambio: dispara el build por polling.
`git log --oneline`	Historial compacto con los hashes que van en la tabla de defectos.

Anexo B — Pantallas de Jenkins y dónde encontrarlas

Pantalla	Cómo llegar	Para qué la usaste
Tools	Manage Jenkins → Tools	Declarar `Maven-3.9` con instalación automática.
Plugins	Manage Jenkins → Plugins	Ver instalados, instalar los que falten.
Credentials	Manage Jenkins → Credentials	Tokens de GitHub o SonarQube (nunca en el `Jenkinsfile`).

Pantalla	Cómo llegar	Para qué la usaste
New Item	Panel principal	Crear jobs Pipeline y Multibranch Pipeline.
Configure	Página del job	Pipeline script from SCM, URL, rama, Script Path.
Stage View	Página del job	Historial de builds por etapa, con colores y duraciones.
Console Output	Página del build	Salida completa de cada paso. Primer lugar donde mirar.
Test Result	Página del build	Pruebas ejecutadas, fallidas, y el detalle de cada una.
Changes	Página del build	Commits incluidos en ese build.
Git Polling Log	Página del job	Cada consulta del polling y si encontró cambios.

Anexo C — Sintaxis del Jenkinsfile y de Maven usada

Expresión	Qué es
`sh 'mvn -B test'`	Ejecuta un comando en un agente Linux o macOS. Si el comando devuelve un código distinto de 0, la etapa falla.
`bat 'mvn test'`	Lo mismo en un agente Windows. En el contenedor de esta práctica no funciona.
`junit 'target/surefire-reports/*.xml'`	Lee los informes XML de Surefire, publica Test Result y marca el build UNSTABLE si hay fallos.
`archiveArtifacts artifacts: 'target/*.jar', fingerprint: true`	Guarda el `.jar` con el build y registra su huella para rastrearlo.
`maven 'Maven-3.9'` (en `tools`)	Pone en el PATH la instalación de Maven con ese nombre exacto.
`pollSCM('H/2 * * * *')`	Revisa el repositorio cada 2 minutos. Campos: minuto, hora, día del mes, mes, día de la semana.
`post { always { } success { } unstable { } failure { } }`	Bloques que se ejecutan al final según el resultado.
`timestamps()`, `timeout(time: 15, unit: 'MINUTES')`	Opciones: marcas de tiempo en la consola y límite de duración.
`-Dmaven.test.failure.ignore=true`	Maven termina con éxito aunque fallen pruebas. Solo es aceptable si un paso `junit` posterior marca el build.
Patrones de Surefire	Por defecto ejecuta `**/Test*.java`, `**/*Test.java`, `**/*Tests.java` y `**/*TestCase.java`. Cualquier otro nombre se ignora sin aviso.

En las prácticas siguientes este pipeline crece: a las etapas de hoy se suman el análisis de SonarQube de la Práctica 3, la suite de Cypress de la Práctica 4 y la prueba de carga de la Práctica 5, y el `.jar` que hoy solo se archiva pasa a desplegarse en un entorno de staging. El `Jenkinsfile` que escribiste hoy es la base sobre la que se construye todo eso.